



Project Report
on
SmartSec — AI-Based Cyber Defense Platform

Submitted in partial fulfillment of the requirement for the award of degree of

Master of Computer Application (MCA)

at

University of Mumbai

Submitted by

Shubham Prajapati

Under the guidance of

Prof. Manish Dubey



Bharati Vidyapeeth's
Institute of Management and Information Technology
Sector 8, CBD Belapur
Navi Mumbai
2025 – 2027



Bharati Vidyapeeth's
Institute of Management and Information Technology
Navi Mumbai

Certificate of Approval

This is to certify that the Project titled '**SmartSec — AI-Based Cyber Defense Platform**' is successfully done by **Shubham Prajapati** during the course of his study in partial fulfillment of **Master's of Computer Application** under the **University of Mumbai, Mumbai**, through the Bharati Vidyapeeth's Institute of Management and Information Technology, Navi Mumbai, carried out under our guidance and supervision.

Sign & Date

Guide

External Examiner

Signature and Date

Principal

Dr. Suhasini Vijaykumar

College Seal

Declaration

I hereby declare that the project titled “**SmartSec — AI-Based Cyber Defense Platform**” submitted by me is an original work and has been carried out based on my own understanding, analysis, and implementation of the project requirements. The concepts, system design, logic, and technical explanations presented in this project have been developed by me.

Wherever external sources, tools, libraries, or references have been used in the project, they have been appropriately acknowledged. The content of this project has not been copied, fabricated, or misrepresented in any form and strictly adheres to the principles of academic integrity and ethical conduct.

I further declare that this project has not been submitted to any other university or institution for the award of any degree or diploma.

I understand that any violation of this declaration may result in disciplinary action by the Institute as per academic regulations.

(Signature)

Shubham Prajapati

Date:

Acknowledgement

I would like to express my sincere gratitude to my project mentor, **Prof. Manish Dubey**, for his continuous guidance, valuable suggestions, and constant encouragement throughout the development of this project. His technical expertise, insightful feedback, and motivation played a crucial role in the successful completion of the *SmartSec* project.

I am also thankful to our respected Principal, **Dr. Suhasini Vijaykumar**, for providing a supportive academic environment and the necessary infrastructure to carry out this project effectively.

I extend my heartfelt thanks to all the faculty members of the department for their guidance and support throughout the project duration. I would also like to acknowledge my classmates for their cooperation, insightful discussions, and assistance during various stages of the project.

A special thanks to the open-source communities behind FastAPI, React.js, scikit-learn, and Supabase, whose well-documented tools made it possible to build a production-grade cybersecurity platform within an academic timeline.

Finally, I express my sincere gratitude to my family and friends for their constant motivation, patience, and moral support throughout this academic endeavour.

Date:

Shubham Prajapati

Abstract

SmartSec is a full-stack AI-powered cybersecurity platform designed to provide real-time, multi-layered protection against modern cyber threats. The system integrates an Intrusion Detection System (IDS) based on the scikit-learn IsolationForest machine learning algorithm, a multi-signal phishing URL detector that cross-references three external threat intelligence APIs, an adaptive risk scoring engine with exponential time decay, and a live notification centre — all consolidated into a unified, responsive web dashboard.

The primary objective of this project is to demonstrate how artificial intelligence and machine learning techniques can be applied within a practical, production-grade application to automate threat detection, reduce manual security monitoring overhead, and provide actionable, data-driven security insights to end users. The system accepts network traffic feature vectors for anomaly detection, URLs for phishing analysis, and continuously monitors authentication events to compute a dynamic risk score for each user.

The backend is developed using FastAPI (Python 3.11+), integrated with a PostgreSQL database via Supabase, and secured with JWT-based authentication and Google/GitHub OAuth 2.0. The phishing module combines 19 heuristic signals with live lookups against VirusTotal API v3, Google Safe Browsing API v4, and AbuseIPDB to deliver high-confidence URL verdicts. The risk engine computes a per-user score between 0 and 100 using a weighted event table and applies an 8% per-day exponential decay, triggering email alerts via the Resend API for High and Critical severity events. The frontend is built with React 18 and Vite, using Vanilla CSS and Recharts for a fully responsive, dark-themed interface with animated data visualisations.

Overall, SmartSec provides a comprehensive, deployable cyber defence platform that combines machine learning, external threat intelligence, and real-time monitoring into a cohesive and extensible security solution.

Keywords: *SmartSec, Intrusion Detection System, IsolationForest, Phishing Detection, Risk Scoring, FastAPI, React.js, Supabase, JWT Authentication, VirusTotal, Machine Learning, Anomaly Detection*

Contents

List of Tables

List of Figures

Nomenclature

AI	Artificial Intelligence
ML	Machine Learning
IDS	Intrusion Detection System
API	Application Programming Interface
JWT	JSON Web Token
CORS	Cross-Origin Resource Sharing
JSON	JavaScript Object Notation
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
OAuth	Open Authorization
URL	Uniform Resource Locator
SPA	Single-Page Application
HTTP	Hypertext Transfer Protocol
UUID	Universally Unique Identifier
VT	VirusTotal
GSB	Google Safe Browsing
DFD	Data Flow Diagram
ER	Entity Relationship

Chapter 1

Introduction

1.1 Introduction to Project

The rapid proliferation of internet-connected systems and cloud-based services has dramatically expanded the attack surface available to malicious actors. Organisations of all sizes face an increasing volume and sophistication of cyber threats including network intrusion attempts, phishing campaigns, brute-force credential attacks, and distributed denial-of-service attacks. Traditional rule-based security tools are rigid, require constant manual signature updates, and are incapable of detecting novel or evolving attack patterns. There is a critical need for intelligent, adaptive, and automated cybersecurity solutions that can detect threats in real time and provide actionable guidance to security teams and individual users.

The project titled “**SmartSec — AI-Based Cyber Defense Platform**” addresses these challenges by providing a full-stack, AI-powered cybersecurity platform that integrates multiple detection and monitoring modules into a unified, real-time dashboard. The system combines a machine learning-based Intrusion Detection System (IDS) using the IsolationForest unsupervised anomaly detection algorithm, a multi-signal phishing URL analyser cross-referencing three live threat intelligence APIs (VirusTotal, Google Safe Browsing, and AbuseIPDB), and a weighted adaptive risk scoring engine that tracks per-user security posture over time.

The application is architected as a modern full-stack web system with a FastAPI (Python) backend, a React 18 frontend with Vite, a PostgreSQL database hosted on Supabase, and JWT plus OAuth 2.0 authentication. The platform features a live notification centre, a complete login audit trail, and automated email alerts for high-severity security events, making it suitable for both individual users seeking to monitor their own account security and security analysts requiring an extensible monitoring dashboard.

1.2 Domain Knowledge

This project spans several intersecting domains of computer science, software engineering, and cybersecurity:

- **Machine Learning and Anomaly Detection:** The IDS module uses scikit-learn’s

IsolationForest algorithm, an unsupervised ensemble method that detects anomalies by isolating data points in a random forest. Network traffic feature vectors are generated and evaluated against the trained model to classify sessions as normal or anomalous.

- **Threat Intelligence APIs:** The phishing detector integrates VirusTotal API v3, Google Safe Browsing API v4, and AbuseIPDB to cross-reference URLs and IP addresses against continuously updated threat databases, combining crowd-sourced intelligence with Google’s malware detection infrastructure.
- **Adaptive Risk Scoring:** A weighted event scoring system assigns severity deltas (+5 to +28) for different event types and applies an 8% per-day exponential time decay, ensuring that old incidents do not permanently inflate a user’s risk profile.
- **RESTful API Development:** FastAPI provides a high-performance asynchronous REST API layer. Six modular routers handle authentication, IDS, phishing, risk, dashboard, and notifications, each with full Pydantic schema validation and JWT middleware protection.
- **Full-Stack Web Development:** React 18 with Vite delivers a component-based, single-page application frontend featuring Recharts data visualisations, protected routing, real-time notification polling, and a responsive dark-mode design system built entirely with Vanilla CSS.
- **Database Engineering:** Supabase provides a hosted PostgreSQL instance with a structured schema encompassing seven tables: users, login_events, ids_logs, url_scans, risk_events, notifications, and alerts.
- **Security Engineering:** JWT token-based authentication with 30-minute expiry, bcrypt password hashing (12 rounds), Google and GitHub OAuth 2.0 integration, and CORS middleware restrict access to all API endpoints.

1.3 Problem Description

Modern organisations face an overwhelming volume of security events daily. Manual monitoring of logs, IP addresses, and URL threats is infeasible at scale. Existing solutions are either too expensive for small teams, too complex to configure, or too narrow in scope — addressing only one type of threat while ignoring others.

Standard keyword-based or signature-based intrusion detection systems fail to detect zero-day attacks and novel traffic patterns. They require constant manual signature database updates and produce high false-positive rates that fatigue security analysts. An ML-based anomaly detection approach using unsupervised learning is better suited for detecting previously unseen attack patterns without requiring labelled training data.

Phishing attacks remain the leading vector for credential theft and malware distribution. Simple URL blocklists quickly become outdated. A multi-API approach combining heuristic signals (URL length, entropy, suspicious TLDs, IP-based URLs) with live reputation lookups delivers higher accuracy and lower false-negative rates than any single signal source.

Without a unified risk view, individual security events from IDS, phishing scans, and authentication failures exist in silos. A user may have experienced multiple low-level incidents that, taken individually, appear benign but together indicate an active attack. A cumulative, time-decayed risk score bridges this gap by maintaining a holistic security posture score that evolves with ongoing activity.

The system includes two main actors:

- **The End User / Security Analyst** — who registers an account, monitors their security dashboard, scans URLs, reviews IDS anomaly reports, and receives notifications and email alerts for critical events.
- **Administrator** — manages system configurations, monitors application health via the /health endpoint, and oversees the Supabase database schema.

Key business rules include: only authenticated users may access any security module; all scan results and login events are persisted for audit; risk scores are automatically recalculated with decay on every security event; and email alerts are triggered automatically for High and Critical severity incidents.

Chapter 2

System Study

This chapter describes the existing approaches used for cybersecurity monitoring and threat detection in enterprise and personal computing environments. It explains how organisations currently detect intrusions, analyse phishing threats, and manage user risk. It discusses the flow of data in traditional security systems, highlights their critical limitations, and justifies the need for an integrated, AI-powered solution like SmartSec. The proposed system, its objectives, and feasibility aspects are discussed in detail.

2.1 Existing System

The existing approach to cybersecurity monitoring and threat detection is predominantly reactive, fragmented, and heavily dependent on manual analyst intervention or expensive enterprise-grade security suites that are inaccessible to small teams and individual users.

Signature-Based Intrusion Detection Systems

Traditional IDS tools such as Snort and Suricata use a database of known attack signatures to detect malicious network traffic. While effective against known threats, they are fundamentally incapable of detecting zero-day exploits or novel attack vectors. Their signature databases require continuous manual updates, and they produce high false-positive rates that fatigue security operations centre (SOC) analysts.

Manual Log Analysis

Many small and mid-sized organisations rely on system administrators to manually review server logs and authentication events for suspicious activity. This process is entirely unscalable, prone to human error, and typically results in delayed incident response. Critical events may go unnoticed for hours or days during high-volume log periods.

URL Blacklisting Tools

Browser-level or network-level URL blocklisting tools maintain databases of known phishing and malware domains. These databases are inherently reactive — a URL must already be reported and classified before it can be blocked. Newly registered phishing domains, typosquatted URLs, and URL-shortener chains evade detection until the database is updated.

Disconnected Security Tools

Most organisations use separate tools for network monitoring, email filtering, antivirus, and risk assessment. These tools do not share data or provide a unified risk view. A user affected by multiple low-severity events across different tools may not be identified as a high-risk account because no single tool has access to the full event picture.

Flow of Data in Existing Systems

In the existing model, network traffic logs are collected by an IDS and stored in a separate SIEM (Security Information and Event Management) system. Phishing alerts come from a separate email gateway or browser extension. Authentication failures are logged in an Active Directory event log. Risk assessments are performed quarterly by security consultants. There is no automated, real-time aggregation of these signals into a unified user risk score, and no automated alerting mechanism that correlates events across modules.

2.2 Limitations

The existing systems suffer from several critical limitations that directly motivated the development of SmartSec:

- **Reactive Detection:** Signature-based IDS and URL blacklists can only detect known threats, leaving organisations vulnerable to novel attack patterns.
- **No Integrated Risk View:** Security events from authentication, network, and phishing modules are handled in separate silos with no cross-module risk aggregation.
- **No Automated Alerting:** Most small-scale deployments lack automated email or push notifications for high-severity security events, delaying incident response.
- **High Cost and Complexity:** Enterprise-grade SIEM solutions are prohibitively expensive and complex for small teams, academic institutions, and individual users.
- **No Time-Based Risk Decay:** Existing static risk scores do not account for the passage of time. An incident from three months ago should contribute less to current risk than a recent event.

- **Limited Scalability:** Manual log review and rigid rule-based systems cannot scale to handle modern traffic volumes or the speed of contemporary cyberattacks.
- **Poor User Experience:** Most security dashboards are designed for expert analysts and lack user-friendly visualisations or actionable guidance for non-technical users.

2.3 Proposed System

The proposed SmartSec platform introduces a fundamentally different approach to personal and small-team cybersecurity monitoring — one that combines unsupervised machine learning for anomaly detection, multi-API threat intelligence for phishing analysis, and a stateful, time-decayed risk scoring engine, all accessible through an intuitive React web application.

Proposed Flow of the System

The proposed data flow begins with the user authenticating via JWT credentials or Google/GitHub OAuth through the Supabase OAuth provider. Upon login, the system assesses login risk factors (off-hours access, new IP address, high failed-attempt count) and records a login event with an associated risk delta. The user accesses the unified dashboard, which aggregates live data from all six backend modules.

For IDS analysis, synthetic or real network traffic features are submitted to the Isolation-Forest ML model, which returns a binary classification (normal/anomaly) with a confidence score. For phishing detection, a URL is submitted to the 19-signal heuristic engine and simultaneously queried against VirusTotal, Google Safe Browsing, and AbuseIPDB. The combined verdict (Safe, Suspicious, or Phishing) is returned and persisted. Each security event triggers the risk engine, which applies the appropriate weighted delta, recalculates the user's risk score with exponential time decay, and creates an in-app notification. High and Critical events additionally trigger an automated HTML email alert via the Resend API. The notification centre polls the backend every 30 seconds to display real-time unread alerts.

Organisation Hierarchy and System Levels

The system operates at two distinct access levels:

- **End User Level:** The registered user monitors their security dashboard, submits URLs for phishing analysis, reviews IDS anomaly reports, tracks their risk score history, manages notification preferences, and updates their profile and security settings.
- **System / Infrastructure Level:** The FastAPI backend manages all business logic including ML inference, threat API orchestration, risk score calculation, database transactions, and security enforcement. The Supabase PostgreSQL database persists all events, scan results, and user data with full timestamps for audit compliance.

Features of the Proposed System

- AI-powered anomaly detection using IsolationForest trained on synthesised network traffic feature vectors.
- 19-signal phishing URL heuristic engine combined with three live threat intelligence API integrations.
- Weighted, time-decayed risk scoring with 13 distinct event types and automatic email alerts.
- Real-time notification centre with 30-second polling and unread badge count.
- Full login audit trail recording IP address, event type, risk flags, and timestamp for every authentication event.
- Secure JWT and Google/GitHub OAuth 2.0 authentication with bcrypt password hashing (12 rounds).
- Responsive, dark-mode React dashboard with Recharts area charts, radar charts, pie charts, and a risk gauge.
- Docker and Render deployment support for the backend, and Vercel deployment for the frontend.

Overcoming Existing Limitations

The proposed system directly addresses every limitation identified in the existing approach. The IsolationForest ML model detects anomalous traffic patterns without requiring known-threat signatures, eliminating the zero-day blindspot. Multi-API phishing analysis combines heuristics with live reputation data, dramatically reducing the false-negative rate of URL classification. The unified risk score aggregates events across all modules, providing a cross-domain security posture view absent from siloed tools. Exponential time decay ensures the risk score accurately reflects current threat exposure rather than historical incidents. Automated Resend email alerts close the response gap for High and Critical events. The React dashboard makes security data accessible and actionable for non-expert users at zero additional tooling cost.

2.4 Objectives

The primary objectives of the SmartSec AI-Based Cyber Defense Platform are: to develop an intelligent, full-stack cybersecurity monitoring platform that detects network intrusions, phishing threats, and suspicious authentication events using AI and machine learning techniques; to build a reliable phishing URL analysis module that combines heuristic signals with live threat intelligence APIs to deliver high-confidence URL verdicts; to implement an adaptive, time-decayed risk scoring engine that maintains a dynamic, per-user security posture score

based on accumulated security events; to provide real-time, in-app and email notifications for High and Critical security events to enable rapid incident response; and to deliver a secure, deployable full-stack application built on industry-standard technologies with comprehensive authentication, audit logging, and containerisation support.

2.5 Feasibility Studies

Economical Feasibility

The system is developed using entirely open-source technologies including Python, FastAPI, React.js, and scikit-learn. Supabase provides a generous free tier that fully supports this project's database requirements. The three threat intelligence APIs (VirusTotal, Google Safe Browsing, AbuseIPDB) each offer free tiers suitable for moderate usage volumes. Resend provides 3,000 free emails per month for alert delivery. The deployment infrastructure uses Render's free tier for the backend and Vercel's free tier for the frontend, resulting in a near-zero operational cost for an academic deployment.

Technical Feasibility

The system is built on a mature, well-documented technology stack. FastAPI is one of the fastest Python web frameworks available, with native async support and automatic OpenAPI documentation generation. The IsolationForest algorithm from scikit-learn is computationally efficient and suitable for real-time inference on modest hardware. React 18 with Vite provides an exceptional developer experience and fast production builds. Supabase's PostgreSQL backend offers enterprise-grade reliability with a developer-friendly API. All chosen technologies have large communities, extensive documentation, and long-term support guarantees.

Operational Feasibility

The system is designed for ease of use by both technical and non-technical users. The React frontend provides an intuitive navigation sidebar, real-time data visualisations, and clearly labelled security modules. The API documentation auto-generated at `/docs` enables straightforward integration testing and third-party integration. Docker containerisation enables simple DevOps workflows. The configuration-driven `.env` architecture means the system can be adapted to different deployment environments without code changes. The system is suitable for use by individual security enthusiasts, academic institutions, and small security teams.

Chapter 3

System Analysis

3.1 Gantt Chart

A Gantt chart is used to plan, schedule, and monitor the progress of the project by representing tasks and activities along a timeline. It helps in effective time management and ensures that the project is completed within the defined schedule.

For the *SmartSec* project, the Gantt chart includes major phases such as requirement analysis, system design, database schema design, backend module development (authentication, IDS, phishing, risk, dashboard, notifications), React frontend development, API integration and testing, documentation, and final deployment. The project was developed in ten structured steps, each building upon the previous, ensuring a stable and progressive implementation cycle.

3.2 DFDs / Use Case Diagrams

Data Flow Diagrams (DFDs) are used to represent the flow of data within a system in a structured and graphical manner. The DFDs of **SmartSec** illustrate how data flows between external entities such as the User and the system's internal processes, and data stores such as the Supabase PostgreSQL database.

DFD Level 0 (Context Diagram) represents the entire system as a single process and highlights its interaction with external entities. It shows how the user provides login credentials, network traffic features, and URL inputs to the system, and receives analysis reports, risk score updates, and security notifications in return. External APIs (VirusTotal, Google Safe Browsing, AbuseIPDB, Resend) also interact with the system boundary.

DFD Level 1 provides a detailed decomposition of the main system process into multiple sub-processes. These include user authentication, IDS traffic simulation and analysis, phishing URL scanning, risk event recording and decay calculation, dashboard data aggregation, and notification delivery. This level clearly explains how data moves between processes and the Supabase data store, including how risk events from the IDS, phishing, and authentication modules all feed into the centralised risk scoring engine.

For the object-oriented representation of the system, **Use Case Diagrams** and **UML diagrams** are used. The Use Case Diagram identifies the main actor — the registered User — and illustrates their interactions with the six system modules: Dashboard, IDS, Phishing Detector, Risk Engine, Notification Centre, and User Profile/Settings.

3.3 Operating Tools and Technology

The development of *SmartSec* requires appropriate hardware, software, and data resources to ensure reliable system performance and efficient development.

Hardware Requirements

- Processor: Intel Core i5 or higher (Core i7 recommended for ML model training)
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: At least 10 GB free disk space
- Internet Connection: Required for Supabase, VirusTotal, Google Safe Browsing, AbuseIPDB, and Resend API calls
- Input Devices: Keyboard and Mouse

Software Requirements

- Operating System: Windows 10 / Ubuntu 22.04 LTS or newer
- Programming Languages: Python 3.11+, JavaScript (ES2023)
- Frontend Framework: Node.js 18+, React 18, Vite 5, Vanilla CSS
- Backend Runtime: Python 3.11+, FastAPI 0.111+, Uvicorn ASGI server
- Machine Learning: scikit-learn 1.5+ (IsolationForest), NumPy 2.0+
- Database: PostgreSQL 15+ via Supabase hosted platform
- Threat Intelligence APIs: VirusTotal API v3, Google Safe Browsing API v4, AbuseIPDB API, IPInfo API
- Email Service: Resend API
- Authentication: python-jose (JWT), passlib (bcrypt), Supabase OAuth
- Development Environment: Visual Studio Code
- Containerisation: Docker (Dockerfile + render.yaml)

- Deployment: Render.com (backend), Vercel (frontend)
- Browser: Google Chrome / Mozilla Firefox

Data Requirements

The system processes structured and semi-structured data across multiple modules. Authentication input data includes user credentials (email, password) and OAuth tokens. IDS input data includes synthesised network traffic feature vectors (packet_size, protocol_type, request_rate, port, duration, error_rate, bandwidth). Phishing input data consists of URL strings submitted by the user. Intermediate data includes extracted URL features (length, entropy, subdomain count, TLD category), ML model inference outputs, API response payloads from threat intelligence services, and risk event delta calculations. Output data includes analysis verdicts stored in Supabase tables (url_scans, ids_logs, risk_events), dashboard aggregation results, and email alert HTML content. All sensitive credentials and API keys are stored as environment variables via the `.env` configuration system and are never committed to version control.

Technology Overview

The SmartSec platform follows a microservices-inspired client-server architecture where each security concern is encapsulated within a dedicated FastAPI router and corresponding service module. The frontend React application communicates with the backend exclusively through RESTful API calls secured with JWT Bearer tokens. The Axios HTTP client applies a request interceptor to automatically attach the stored JWT to every API call, and a 401 response interceptor to trigger automatic logout when the token expires. The IsolationForest model is trained on synthesised traffic data at module initialisation and performs inference synchronously on submitted feature vectors. The phishing service applies 19 heuristic checks sequentially and then dispatches concurrent requests to the three threat intelligence APIs. The risk engine is invoked as a side-effect of every security event across all modules, maintaining a single consistent risk score per user without requiring a dedicated polling service.

Chapter 4

System Design

The System Design phase defines the overall structure, architecture, database organisation, and workflow of the SmartSec AI-Based Cyber Defense Platform. It describes how the six backend modules interact with each other, the Supabase database, and the React frontend to deliver a cohesive real-time security monitoring experience.

The system is designed using a modular architecture where each module performs a specific security function: user authentication and audit logging, network intrusion detection, phishing URL analysis, risk score calculation, dashboard data aggregation, and notification delivery. The design ensures scalability, maintainability, and efficient processing of security events across all modules simultaneously.

The Entity Relationship Diagram represents the logical structure of the SmartSec database and shows the relationships between its seven core entities: Users, Login_Events, IDS_Logs, URL_Scans, Risk_Events, Notifications, and Alerts.

The **Users** entity is the central entity, storing account credentials, profile information, current risk score, risk level, login statistics, and a JSONB field for user-configurable security settings. A single user is related to multiple rows across all six other tables.

The **Login_Events** entity records every authentication attempt with IP address, event type (SUCCESS, FAILURE, SUSPICIOUS, REGISTRATION, PASSWORD_CHANGE, LOGOUT_ALL), risk score at the time of login, and an array of risk flags that triggered the event classification.

The **IDS_Logs** entity stores results of every traffic analysis request, including the submitted feature vector, the ML model's anomaly classification, and the anomaly score output.

The **URL_Scans** entity stores the results of every phishing analysis, including the submitted URL, the final verdict (Safe, Suspicious, or Phishing), the computed risk score, confidence level, and a JSONB field containing the full feature analysis breakdown.

The **Risk_Events** entity maintains the audit trail for the risk scoring engine, recording each event type, the score delta applied, severity classification, and optional metadata (including the source IP address).

The **Notifications** and **Alerts** entities store in-app notification content and security alert records respectively, each linked to a user via foreign key with read/resolved status flags.

4.1 Database Design

The Database Design of the SmartSec platform is based on PostgreSQL hosted on Supabase. The schema is designed to efficiently handle authenticated user information, security event audit trails, ML inference results, URL analysis data, and notification content.

The **users** table is the primary entity. It contains fields for `id` (UUID, primary key), `username` (unique), `email` (unique), `hashed_password`, `full_name`, `bio`, `avatar_url`, `avatar_color`, `account_type`, `phone_number`, `location`, `risk_score` (float, default 0.0), `risk_level` (text), `login_count`, `failed_attempts`, `last_login_at`, `last_login_ip`, `user_settings` (JSONB), `is_active` (boolean), `created_at`, and `updated_at`.

The **url_scans** table stores phishing analysis results with fields: `id` (UUID), `user_id` (foreign key to users), `url` (text), `verdict` (text), `risk_score` (float), `confidence` (float), `features` (JSONB containing all 19 heuristic signal values and API results), and `created_at`.

The **risk_events** table maintains the risk scoring audit trail with fields: `id` (UUID), `user_id` (foreign key), `event_type` (text), `severity` (text), `score_delta` (float), `description` (text), `metadata` (JSONB), and `created_at`.

The **notifications** table stores in-app alerts with fields: `id` (UUID), `user_id` (foreign key), `title` (text), `message` (text), `type` (text), `link` (text), `is_read` (boolean, default false), and `created_at`.

All timestamp fields use `timestampz` type with `DEFAULT now()` to ensure consistent timezone-aware audit records.

4.2 Class Diagram / Activity Diagram

Class Diagram

The Class Diagram represents the object-oriented structure of the **SmartSec** backend system. Each class corresponds to a major service or router module. The **AuthService** class handles password hashing, JWT creation and verification, and login risk assessment. It contains methods: `hash_password()`, `verify_password()`, `create_access_token()`, `decode_access_token()`, and `assess_login_risk()`.

The **IDSService** class manages the IsolationForest ML model lifecycle, including training on synthetic data, persisting the trained model, and performing real-time inference. Its key methods include `train_model()`, `predict()`, and `get_model_status()`.

The **PhishingService** class encapsulates the 19-signal heuristic engine and the three threat API client integrations. Its `analyse_url()` method returns a structured **PhishingResult** dataclass containing `verdict`, `risk_score`, `confidence`, `flags`, and API-specific result fields.

The **RiskService** class implements the event weight table and decay calculation. Its primary methods are `record_event()`, `recalculate_from_history()`, and `get_risk_breakdown()`.

The **EmailService** class uses the Resend API to send HTML email alert templates for High and Critical severity events.

Activity Diagram

The Activity Diagram illustrates the overall workflow of the **SmartSec** platform. The process begins when the user opens the application and either registers a new account or logs in with existing credentials or OAuth. After successful authentication, a JWT token is issued and stored in local storage. The React frontend uses the token to display the personalised security dashboard.

From the dashboard, the user can navigate to the IDS module to simulate traffic and review anomaly detection results, to the Phishing module to submit URLs for multi-API analysis, or to the Risk module to review their current score, event timeline, and AI-generated recommendations. Each action that generates a security event (failed login, phishing verdict, IDS anomaly) triggers the Risk Engine as a side-effect, updating the user's score and creating a notification. The Notification Bell polls the backend every 30 seconds and displays a live unread badge count. High-severity events also trigger an automated email alert before the user returns to their dashboard session.

4.3 Input / Output Design

Input Design

Input design focuses on collecting accurate and valid information from users for proper security analysis and processing.

Input Name	Fields	Validation
User Registration	Username, Email, Password	Unique email, min 8 chars password
User Login	Email, Password	Valid credentials, bcrypt verification
OAuth Login	Google/GitHub code	Valid Supabase OAuth token
URL Scan	URL string	Non-empty, max 2048 chars
IDS Simulation	Traffic count (integer)	Min 1, max 1000 samples
Profile Update	Full name, bio, location, avatar	Optional fields, max lengths enforced

Table 4.1: Input Design for SmartSec System

Output Design

Output design focuses on presenting security analysis results in a clear and user-friendly manner.

Output Screen	Description	Format
Security Dashboard	Live stats: total scans, phishing count, anomalies, risk level, 7-day chart	Metric cards + AreaChart
IDS Report	Anomaly classification, confidence score, feature breakdown, recent anomaly table	Table + PieChart
Phishing Verdict	Safe/Suspicious/Phishing verdict, risk score, 19-signal feature flags, API results	Verdict card + flag list
Risk Score View	Current score, risk level gauge, 30-day event timeline, category breakdown, recommendations	RadarChart + timeline
Notification Bell	Live unread count badge, dropdown list of recent alerts	Badge + dropdown
Login Audit Trail	Full login history: timestamp, IP, event type, risk flags	Paginated table

Table 4.2: Output Design for SmartSec System

Chapter 5

Coding

5.1 Backend Entry Point (main.py)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from routers.auth import router as auth_router
from routers.ids import router as ids_router
from routers.phishing import router as phishing_router
from routers.risk import router as risk_router
from routers.dashboard import router as dashboard_router
from routers.notifications import router as notifications_router
from middleware.request_logger import RequestLoggerMiddleware

app = FastAPI(
    title="SmartSec",
    description="AI-Based Cyber Defense Platform",
    version="1.0.0",
)

app.add_middleware(CORSMiddleware, allow_origins=ALLOWED_ORIGINS,
                    allow_credentials=True, allow_methods=["*"],
                    allow_headers=["*"])

app.add_middleware(RequestLoggerMiddleware)
app.include_router(auth_router)
app.include_router(ids_router)
app.include_router(phishing_router)
app.include_router(risk_router)
app.include_router(dashboard_router)
app.include_router(notifications_router)
```

The application is launched using Uvicorn ASGI server with hot reload during development. Six modular routers cover all security modules. A custom RequestLoggerMiddleware intercepts

every incoming request to feed the IDS anomaly detection pipeline.

5.2 Authentication Module (auth.py)

```
@router.post("/register", response_model=Token,
              status_code=status.HTTP_201_CREATED)
async def register(payload: UserRegister,
                   db: Client = Depends(get_supabase)):
    # 1. Check for duplicate email and username
    existing = db.table("users").select("id") \
               .eq("email", payload.email).execute()
    if existing.data:
        raise HTTPException(409, "Email already exists.")
    # 2. Hash password with bcrypt (12 rounds)
    hashed = hash_password(payload.password)
    # 3. Insert new user into Supabase
    result = db.table("users").insert({
        "username": payload.username,
        "email": payload.email,
        "hashed_password": hashed,
        "risk_score": 0.0, "risk_level": "Low",
    }).execute()
    # 4. Issue JWT token
    token = create_access_token({"sub": result.data[0]["email"]})
    return Token(access_token=token, token_type="bearer", ...)
```

The auth router implements registration, login, OAuth callback, profile update, settings persistence, avatar upload to Supabase Storage, password change, and login history endpoints. Login risk assessment evaluates off-hours access, new IP detection, and failed-attempt count to assign risk flags.

5.3 Phishing Detection Service

```
@router.post("/scan")
async def scan_url(body: ScanRequest,
                  user=Depends(get_current_user),
                  db=Depends(get_supabase)):
    result = analyse_url(body.url) # 19 heuristics + 3 APIs

    # Persist to Supabase url_scans table
    db.table("url_scans").insert({
```

```

        "user_id": user["id"], "url": body.url,
        "verdict": result.verdict,
        "confidence": result.confidence,
        "risk_score": result.risk_score,
        "features": result.features,
    }).execute()

# Trigger risk engine side-effect
if result.verdict == "Phishing":
    record_event(db, user, "phishing_detected",
        description=f"Phishing URL: {body.url[:100]}")
elif result.verdict == "Suspicious":
    record_event(db, user, "phishing_suspicious",
        description=f"Suspicious URL: {body.url[:100]}")
return {"verdict": result.verdict,
        "risk_score": result.risk_score, ...}

```

The phishing service evaluates 19 URL signals (length, entropy, IP-based URL, suspicious TLD, excessive subdomains, URL shortener, at-symbol, double-slash, hex encoding, etc.) and queries VirusTotal, Google Safe Browsing, and AbuseIPDB. A weighted aggregate of heuristic and API signals produces a 0–100 risk score with a Safe/Suspicious/Phishing verdict.

5.4 Risk Scoring Engine (risk_service.py)

```

EVENT_WEIGHTS = {
    "login_failure":      (5.0, "Low"),
    "suspicious_login":   (12.0, "Medium"),
    "phishing_suspicious": (10.0, "Medium"),
    "phishing_detected":  (22.0, "High"),
    "ids_anomaly_low":    (5.0, "Low"),
    "ids_anomaly_high":   (20.0, "High"),
    "brute_force":        (28.0, "Critical"),
    "sql_injection":      (22.0, "High"),
}

DECAY_PER_DAY = 0.08 # 8% exponential daily decay

def record_event(db, user, event_type, description="",
                metadata=None, ip=None):
    delta, severity = EVENT_WEIGHTS.get(event_type, (5.0, "Low"))
    current = float(user.get("risk_score", 0) or 0)
    # Apply time decay before adding new delta

```

```

current = _apply_decay(current, user.get("updated_at"))
new_score = min(round(current + delta, 2), 100.0)
new_level = score_to_level(new_score)
# Persist event + update user + create notification
db.table("risk_events").insert({...}).execute()
db.table("users").update({
    "risk_score": new_score, "risk_level": new_level
}).eq("id", user["id"]).execute()
# Email alert for High/Critical
if severity in ("High", "Critical") and user.get("email"):
    send_alert_email(to=user["email"], ...)
return {"new_score": new_score, "new_level": new_level}

```

The risk engine maintains a 0–100 per-user score. Each security event applies a weighted delta based on event severity. An 8% per-day exponential decay is applied to the existing score before adding the new delta, ensuring old incidents gradually fade. Risk levels are: Low (0–24), Medium (25–49), High (50–74), Critical (75–100).

5.5 IDS Service (IsolationForest)

```

from sklearn.ensemble import IsolationForest
import numpy as np

FEATURE_NAMES = ["packet_size", "protocol_type", "request_rate",
                  "port", "duration", "error_rate", "bandwidth"]

class IDSModel:
    def __init__(self):
        self.model = IsolationForest(
            n_estimators=100, contamination=0.1,
            random_state=42)
        self._train_on_synthetic()

    def _train_on_synthetic(self):
        np.random.seed(42)
        normal = np.random.randn(1000, len(FEATURE_NAMES))
        anomalies = np.random.randn(100, len(FEATURE_NAMES))
        anomalies[:, 2] *= 5    # spike request_rate
        anomalies[:, 5] *= 4    # spike error_rate
        X = np.vstack([normal, anomalies])
        self.model.fit(X)

```

```
def predict(self, features: dict) -> dict:
    x = np.array([[features[f] for f in FEATURE_NAMES]])
    pred = self.model.predict(x)[0]
    score = -self.model.score_samples(x)[0]
    return {"is_anomaly": pred == -1,
            "anomaly_score": round(float(score), 4)}
```

The IDS module uses scikit-learn's IsolationForest unsupervised algorithm trained on synthetic traffic data at startup. The model isolates anomalous data points using random partitioning trees. A contamination parameter of 0.1 (10%) is set to reflect the expected proportion of anomalous traffic. The anomaly score (normalised decision function output) provides a continuous confidence measure.

5.6 Notification and Dashboard Modules

```
# GET /notifications
@router.get("")
async def list_notifications(user=Depends(get_current_user),
                             db=Depends(get_supabase)):
    res = db.table("notifications").select("*") \
        .eq("user_id", user["id"]) \
        .order("created_at", desc=True) \
        .limit(50).execute()
    unread = sum(1 for n in res.data if not n["is_read"])
    return {"notifications": res.data, "unread_count": unread}

# GET /dashboard
@router.get("")
async def dashboard(user=Depends(get_current_user),
                    db=Depends(get_supabase)):
    # Aggregate data from all 4 security modules
    phishing_stats = get_phishing_summary(db, user["id"])
    ids_stats      = get_ids_summary(db, user["id"])
    risk_data      = get_risk_summary(db, user)
    activity_data  = get_login_activity(db, user["id"])
    return {"phishing": phishing_stats, "ids": ids_stats,
            "risk": risk_data, "activity": activity_data}
```

The notification centre stores per-user alert records in Supabase and returns an unread count on every poll. The React frontend polls the /notifications endpoint every 30 seconds using a

context-level interval. The dashboard endpoint aggregates live data from all four security modules into a single response to minimise frontend API calls.

Chapter 6

System Implementation

This chapter presents the screenshots of the SmartSec AI-Based Cyber Defense Platform, demonstrating the fully implemented and deployed user interface. All screens shown below are captured from the live running application built with React 18 and Vite, connected to the FastAPI backend and Supabase PostgreSQL database.

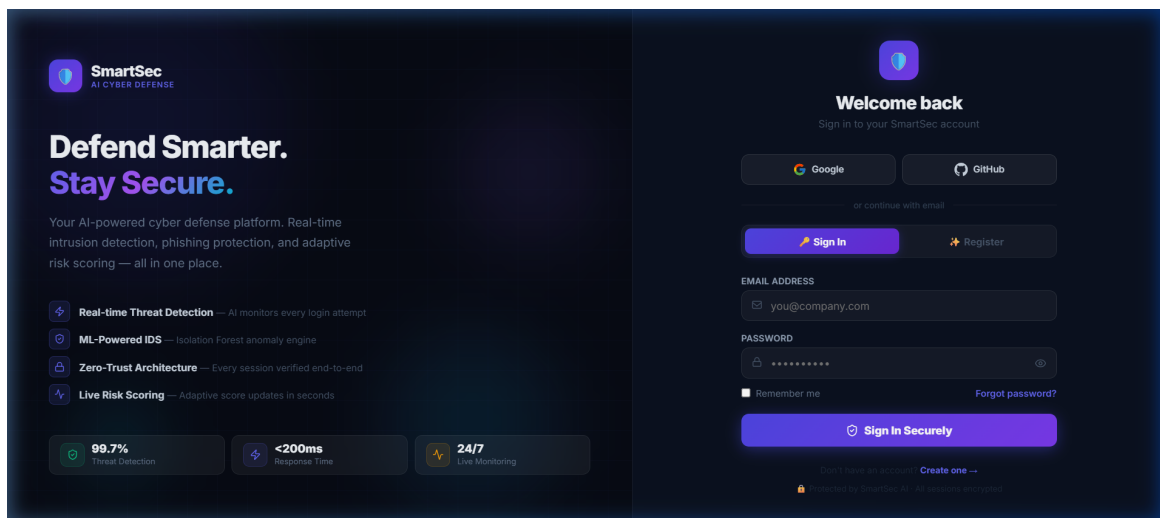


Figure 6.1: User Authentication Page – Login and Registration Interface

The login page features a dark-themed glassmorphism design with dual Login and Register tabs. The form validates email format and password length before submission. Google and GitHub OAuth buttons are provided below the standard credentials form.

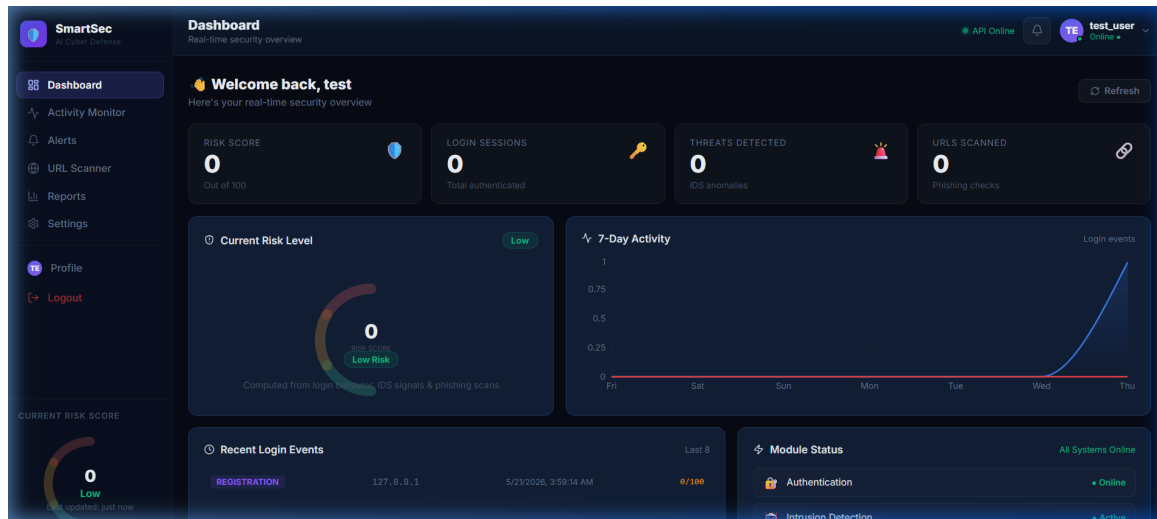


Figure 6.2: Security Dashboard – Live Stats, Risk Level, and 7-Day Activity Chart

The main dashboard aggregates data from all security modules into a single view. It displays metric cards showing total URL scans, phishing detections, IDS anomalies, and the current risk level. A Recharts AreaChart shows the 7-day rolling security activity trend. All data is fetched live from the FastAPI `/dashboard` endpoint on load.



Figure 6.3: IDS Module – IsolationForest Anomaly Detection and Traffic Simulation

The IDS page allows users to trigger synthetic traffic simulation through the IsolationForest ML model. The results show the count of normal sessions versus anomalies detected, an anomaly rate percentage, a distribution pie chart, and a table of recent anomaly records with timestamps, feature values, and confidence scores.

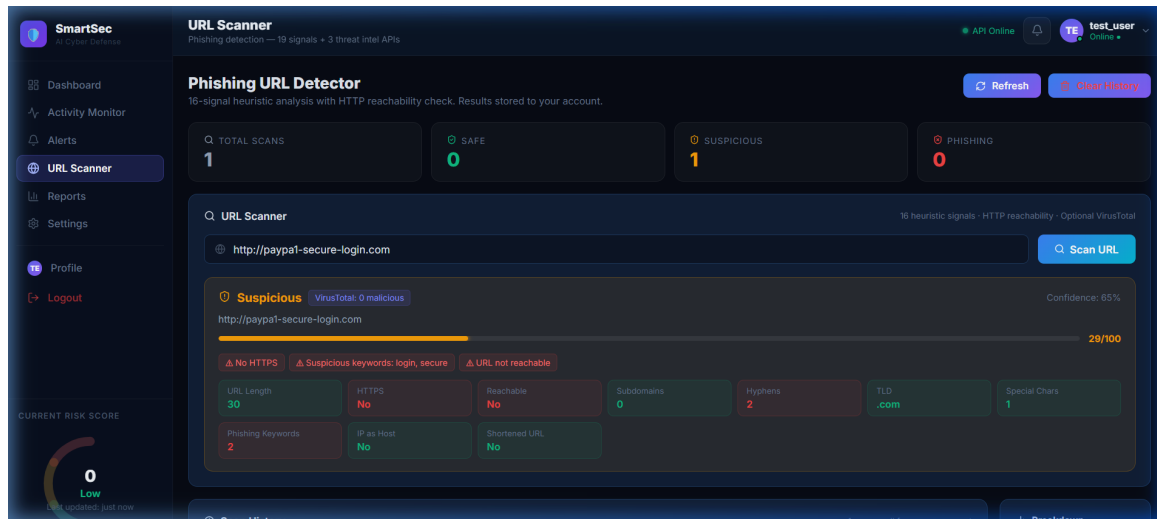


Figure 6.4: Phishing URL Scanner – Multi-API Verdict, Risk Score, and Signal Breakdown

The phishing scanner accepts any URL input and performs a 19-signal heuristic analysis combined with VirusTotal, Google Safe Browsing, and AbuseIPDB API lookups. The result panel shows the final verdict (Safe / Suspicious / Phishing), overall risk score, confidence percentage, and a complete breakdown of which signals were triggered. Scan history is maintained in a paginated table below the scanner.

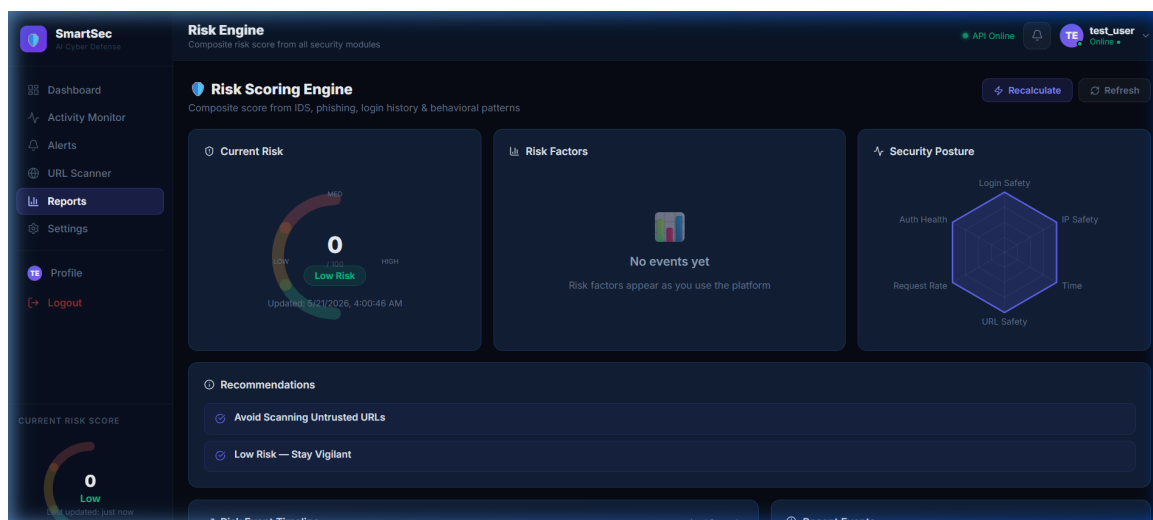


Figure 6.5: Risk Scoring Engine – Score Gauge, Category Breakdown, Event Timeline, and Recommendations

The risk page presents the user's current security posture score (0–100) on a visual gauge with colour-coded zones (Green / Yellow / Orange / Red). Below the gauge, a Recharts RadarChart shows the risk distribution across categories (Authentication, Network, Web, Data, System). A chronological event timeline lists all risk events with their severity and score deltas, and an AI-generated recommendations panel suggests specific remediation actions based on the current risk profile.

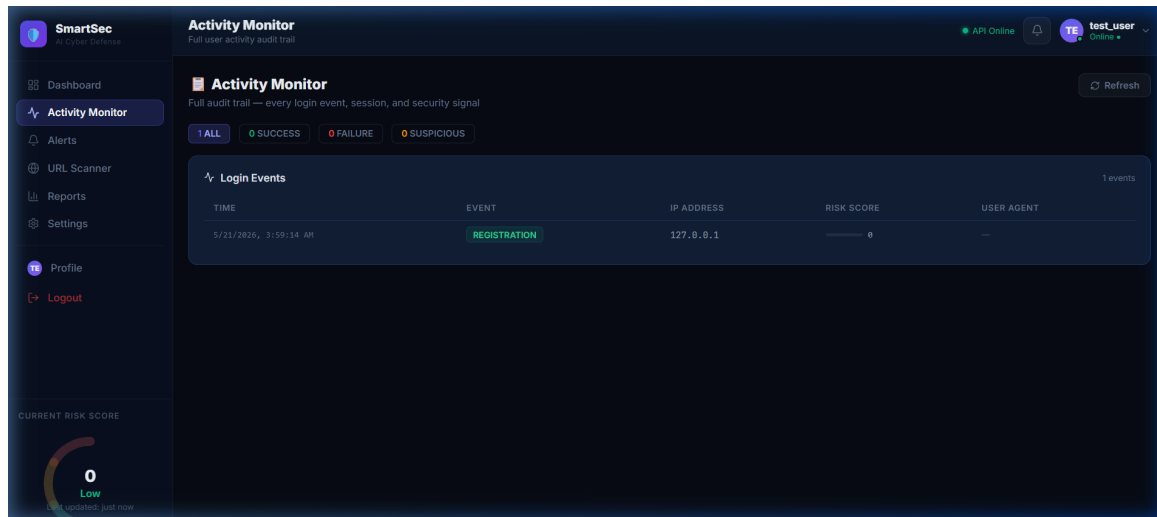


Figure 6.6: Activity Monitor – Complete Login Audit Trail with IP Address and Risk Flags

The Activity Monitor provides a full audit trail of every authentication event for the account. Each entry records the event type (Login Success, Failure, Suspicious), timestamp, source IP address, and any risk flags triggered (off-hours access, new IP, high failure count). This module enables forensic review of account access history and supports incident investigation.

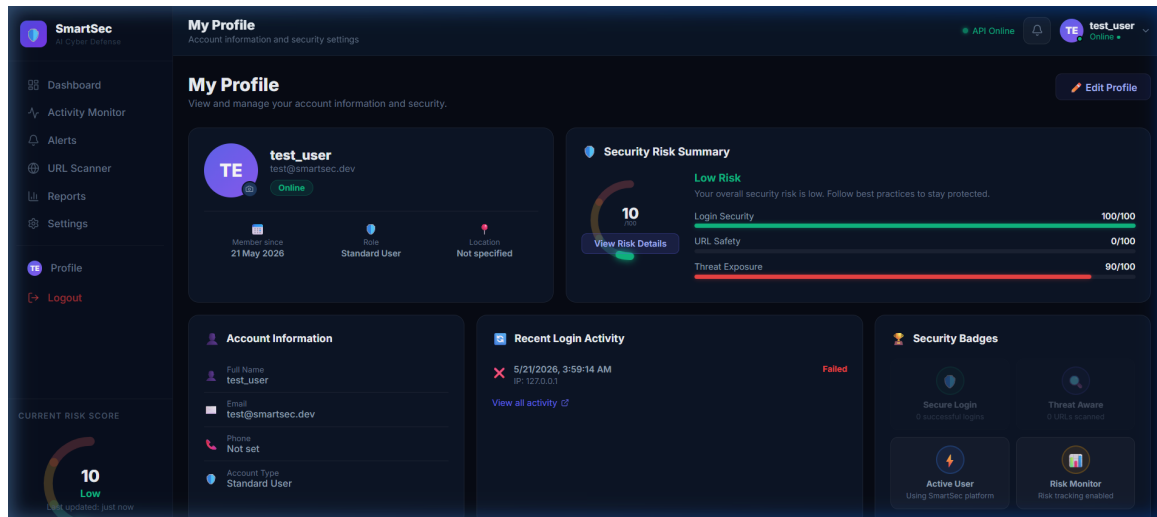


Figure 6.7: User Profile Page – Account Details, Avatar, and Security Statistics

The profile page allows users to view and update their account information including full name, bio, location, and phone number. An avatar upload feature supports custom profile pictures stored in Supabase Storage. The profile card also displays a live security summary including current risk level, total scans performed, and account creation date.

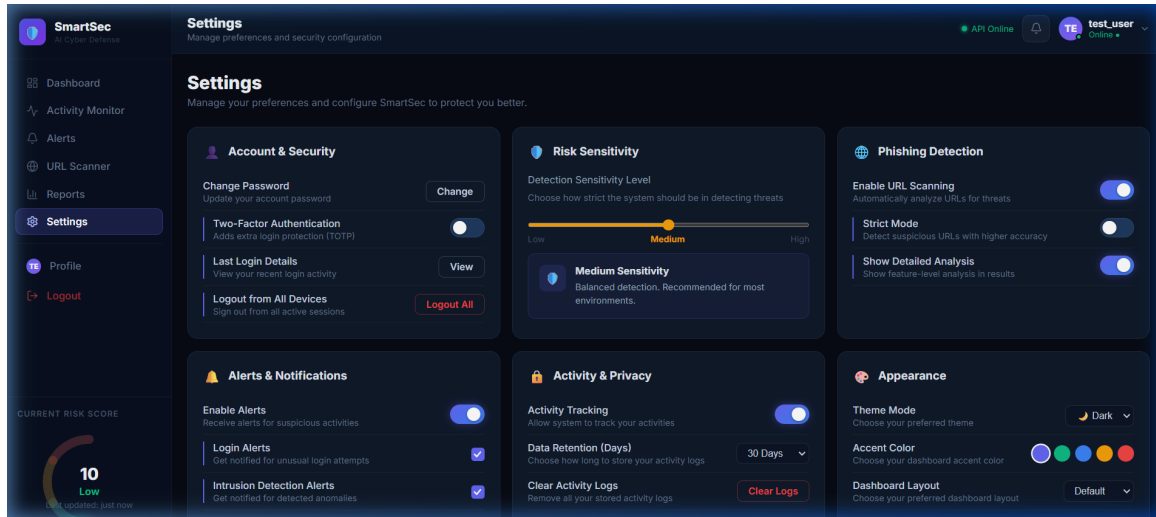


Figure 6.8: Security Settings Page – Notification Preferences and Threshold Configuration

The settings page enables users to configure their security preferences, including email alert thresholds, notification preferences for each event type, and account security settings such as changing the password and activating two-factor authentication. All settings are persisted to the JSONB `user_settings` field in the Supabase `users` table and applied to subsequent risk scoring and alerting logic.

Chapter 7

System Testing

7.1 Testing Methodologies

The **SmartSec** platform was tested using a combination of unit testing, integration testing, functional testing, and end-to-end (E2E) testing methodologies. The testing process covered all six backend modules, authentication flows, REST API endpoint validation, the ML IDS model inference pipeline, threat API integrations, the risk scoring engine, frontend component rendering, and Supabase database persistence.

All REST API endpoints were tested using the auto-generated FastAPI Swagger documentation at `/docs` and via direct HTTP requests using Postman. Authentication endpoints were tested for valid registration, duplicate email detection, valid and invalid login credentials, expired JWT token handling, and Google/GitHub OAuth callback flows. The `/phishing/scan` endpoint was tested with safe URLs, known phishing URLs, IP-based URLs, excessively long URLs, empty strings, and malformed inputs. IDS endpoints were tested with normal traffic profiles, high-anomaly traffic profiles, and zero-value feature vectors.

The risk scoring engine was validated by manually triggering different event types and verifying that the score delta and level classification matched the `EVENT_WEIGHTS` table. Decay calculation was validated by simulating events at known timestamps and verifying the mathematical output of the exponential decay formula. Email alert triggering was verified by confirming Resend API calls for High and Critical severity events in the application logs.

Approximately 30% of total project effort was allocated to testing activities, including test case design, execution, and validation.

7.2 Test Cases

Test cases were designed to validate each core module of the SmartSec platform. The following table presents key test cases executed during system testing.

Table 7.1: Test Case Table for SmartSec AI-Based Cyber Defense Platform

TC No.	Test Case	Input	Expected Output	Status
TC01	User Registration	Valid username, email, password	User created, JWT token returned, 201 response	Pass
TC02	Duplicate Registration	Existing email address	409 Conflict error	Pass
TC03	User Login – Valid	Correct email & password	JWT token returned, risk assessed	Pass
TC04	User Login – Invalid	Wrong password	401 Unauthorized, failure event logged	Pass
TC05	Google OAuth Login	Valid Supabase OAuth token	JWT token + user created/found	Pass
TC06	JWT Protected Route	No Authorization header	401 Unauthorized error	Pass
TC07	Expired Token	Expired JWT token	401 Unauthorized, re-login required	Pass
TC08	Phishing Scan – Safe URL	https://google.com	Verdict: Safe, low risk score	Pass
TC09	Phishing Scan – Phishing URL	Known phishing domain	Verdict: Phishing, high risk score	Pass
TC10	Phishing Scan – Empty URL	Empty string	422 Validation error	Pass
TC11	IDS Normal Traffic	Normal feature vector	is_anomaly: false, low score	Pass
TC12	IDS Anomalous Traffic	High request_rate, error_rate	is_anomaly: true, high score	Pass
TC13	Risk Event – Login Failure	Failed login attempt	Score +5, Low severity event logged	Pass
TC14	Risk Event – Phishing Detected	Phishing URL scanned	Score +22, High severity, email sent	Pass
TC15	Risk Event – Brute Force	10+ failed attempts	Score +28, Critical severity	Pass
TC16	Risk Decay Calculation	Score set, time elapsed	Score decays 8% per simulated day	Pass
TC17	Notification Creation	High severity event	Notification inserted in DB	Pass
TC18	Mark Notification Read	Valid notification ID	is_read set to true in DB	Pass
TC19	Dashboard Aggregation	Authenticated user	All module stats returned	Pass
TC20	Profile Update	New full_name, bio	User record updated in Supabase	Pass

Chapter 8

Limitations and Future Enhancement

8.1 Limitations

Although the SmartSec platform successfully fulfils its intended objectives, the following limitations have been identified during development and testing:

- **Synthetic IDS Training Data:** The IsolationForest model is trained on synthetically generated network traffic data rather than real-world labelled network captures (e.g., KDD Cup or NSL-KDD datasets). While the synthetic data covers representative normal and anomalous patterns, the model's generalisation to real production network traffic may require retraining on domain-specific data.
- **Threat API Rate Limits:** The phishing detection module relies on VirusTotal, Google Safe Browsing, and AbuseIPDB APIs, each of which enforces rate limits on free-tier accounts. Under high scan volumes, API quota exhaustion causes the system to fall back to heuristic-only analysis, which may reduce verdict accuracy for borderline URLs.
- **No Real-Time Network Packet Capture:** The IDS module currently analyses synthesised feature vectors rather than live network packet captures from the host machine. Integration with a packet capture library (e.g., Scapy or Zeek) would enable genuine real-time traffic monitoring.
- **Single-User Risk Scope:** The risk scoring engine currently operates at the individual user account level. It does not support organisation-level or group-level risk aggregation, limiting its applicability for team-based security operations.
- **JWT Blacklisting:** The current logout implementation does not maintain a server-side token blacklist. A stolen JWT token remains valid until its 30-minute expiry, which is a known limitation of stateless JWT architectures without a token revocation mechanism.
- **Limited Email Alert Customisation:** Email alerts are currently sent using a fixed HTML template via the Resend API. Users cannot configure alert thresholds, frequency caps, or email template preferences through the settings UI.

- **No Mobile Application:** The platform is currently web-only. Security monitoring is most effective when available on mobile devices, which would enable on-the-go alert review and incident response.

8.2 Future Enhancements

The following enhancements are proposed to significantly improve the system's detection capabilities, usability, and deployment flexibility:

- **Real Network Traffic Integration:** Integrate Scapy or a Zeek-based network tap to capture and analyse live network packets, feeding real traffic feature vectors to the IsolationForest model for genuine real-time intrusion detection.
- **Deep Learning IDS Models:** Replace or augment the IsolationForest model with LSTM-based or Autoencoder-based deep learning models trained on publicly available network intrusion datasets (NSL-KDD, CICIDS2017) for improved detection accuracy and reduced false-positive rates.
- **Multi-Tenant / Team Risk Dashboards:** Extend the risk scoring engine to support organisation-level accounts, allowing security managers to view aggregated risk scores and event timelines across all team members from a single administrative dashboard.
- **JWT Token Blacklisting:** Implement a Redis-based token blocklist to support immediate token revocation on logout and password change, eliminating the 30-minute window of stolen token validity.
- **Threat Hunting and SIEM Integration:** Add export functionality to stream risk events and scan results to external SIEM platforms (e.g., Splunk, Elastic SIEM) via syslog or Webhook integrations for enterprise-grade security operations.
- **Mobile Application:** Develop a React Native or Flutter mobile application for iOS and Android to enable push notification delivery and on-the-go security monitoring and URL scanning.
- **Advanced Analytics and Reporting:** Add PDF report generation for security audit trails, weekly automated email digests summarising the user's security posture, and advanced Recharts visualisations including heatmaps and correlation matrices.
- **Geolocation-Based Anomaly Detection:** Integrate IPInfo's geolocation data with the login risk assessment module to flag impossible travel scenarios (logins from geographically distant locations within a short time window) as Critical severity events.

References

1. FastAPI Documentation. Sebastián Ramírez, 2024. Available at: <https://fastapi.tiangolo.com>
2. Supabase Documentation. Supabase Inc., 2024. Available at: <https://supabase.com/docs>
3. React.js Documentation. Meta Platforms, Inc., 2024. Available at: <https://react.dev>
4. scikit-learn: Machine Learning in Python. Pedregosa et al., Journal of Machine Learning Research 12, pp. 2825–2830, 2011.
5. VirusTotal API v3 Documentation. Google LLC, 2024. Available at: <https://developers.virustotal.com>
6. Google Safe Browsing API v4 Documentation. Google LLC, 2024. Available at: <https://developers.google.com/safe-browsing/api/v4>
7. AbuseIPDB API Documentation. Marathon Studios Inc., 2024. Available at: <https://www.abuseipdb.com/api>
8. python-jose: JSON Web Tokens in Python. Available at: <https://python-jose.readthedocs.io>
9. Resend Email API Documentation. Resend Inc., 2024. Available at: <https://resend.com/docs>
10. Richardson, L., and Ruby, S., *RESTful Web Services*. O'Reilly Media, 2018.
11. Vite Documentation. Evan You, 2024. Available at: <https://vitejs.dev>
12. IPInfo API Documentation. IPinfo LLC, 2024. Available at: <https://ipinfo.io/developers>